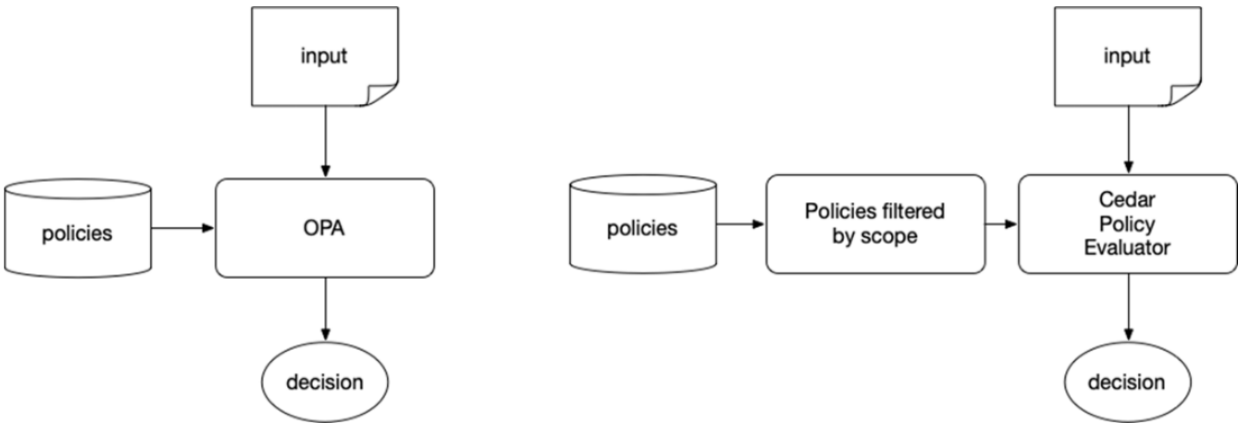


Figure 8.1 OPA (left) vs Cedar (right) policy evaluation flow. OPA evaluates all policies using the full input, with no built-in filtering based on principal, action, or resource. In contrast, Cedar policies can declare structural scope, allowing the engine to filter policies before evaluation. This improves performance and ensures that only relevant policies are considered.

This structural filtering is a key architectural advantage of Cedar. Because policies can specify which principals, actions, and resources they apply to, the Cedar engine can quickly narrow down the set of policies that must be evaluated for each request. This not only boosts performance but also makes policy behavior more predictable and easier to audit, especially important in systems with large, diverse policy sets. Cedar's scoping also supports schema validation, static analysis, and indexing, making policies not only expressive but also safer and more efficient to evaluate.

8.2.3 Applying the PARC model to OPA and Rego

OPA and Rego are designed to be general-purpose: they can evaluate any type of structured decision, not just access control. This flexibility is a strength, but it also means OPA doesn't impose a specific model for how to represent principals, actions, resources, or context. Instead, it's up to the policy author to define the structure of the input object and write rules against it. Let's see how this looks from the perspective of PARC.

- **Principal**—Rego does not have a built-in concept of a principal. Instead, policies operate on a user-defined `input.principal` object—usually a JSON structure included with the request. This may contain attributes like identifiers, groups, or departments, but it's up to the policy author to define them. There's no schema enforcement or